

代码生成论

从 C 到 Rust 到 MLOG

Typst 编译 — 2026-07-16

所有编译器都是翻译器。

从 C 到 x86, 从 Rust 到 LLVM IR, 从 DSL 到 MLOG—
每条翻译链背后都是同一套方法论。

目录

1	序章：代码生成的普遍性	4
1.1	三条翻译链	4
1.2	每一层丢掉什么、添加什么	4
1.3	核心问题	4
1.4	本书结构	4
2	最低抽象 — CPU 看到的世界	5
2.1	寄存器：CPU 的零级缓存	5
2.2	内存：大但慢	5
2.3	核心指令	5
2.4	没有的东西	6
2.5	一个简单的 C → 汇编对照	6
2.6	小结	7
3	C 表达式 → 汇编	8
3.1	最简单的对照	8
3.2	复杂表达式分解	8
3.3	用虚拟寄存器思考	8
3.4	类型如何消失	9
3.5	指针和内存寻址	9
3.6	结构体字段访问	10
3.7	表达式翻译的通用模式	10
3.8	小结	10
4	C 控制流 → 汇编	11
4.1	if-else	11
4.2	while 循环	11
4.3	for 循环	12
4.4	switch	12
4.5	短路求值	13
4.6	控制流图 (CFG)	14
4.7	小结	14
5	C 函数 → 汇编	15
5.1	调用约定的意义	15
5.2	栈帧	15
5.3	call 和 ret 的内部操作	16
5.4	递归的栈展开	16
5.5	尾调用优化	17
5.6	和 MLOG 的对比	17
5.7	小结	17
6	寄存器分配 — 编译器的核心	18
6.1	问题的核心	18
6.2	活跃范围 (Live Range)	18
6.3	图着色 (Graph Coloring)	19
6.4	线性扫描分配	19
6.5	这和 MLOG 有什么关系	19
6.6	小结	19
7	三地址码 — 编译器的通用语言	21
7.1	三地址码的形式	21
7.2	为什么三地址码是“最佳”IR	21
7.3	SSA 形式：更严格的三地址码	22
7.4	MLOG 和 x86 在三地址码下的统一	22

7.5	小结	23
8	从 C 到 Rust — 多了什么	24
8.1	C 编译器已经解决的问题	24
8.2	Rust 多了什么—在编译时	24
8.3	动态分发: trait object	25
8.4	Rust 仍然面临和 C 相同的编译问题	25
8.5	小结	25
9	Rust 特有构造 → 汇编	26
9.1	enum 和 match	26
9.2	闭包	27
9.3	async / .await	27
9.4	所有 Rust 构造的汇编归宿	28
9.5	小结	28
10	优化 — 编译器在帮你做什么	29
10.1	常量折叠 (Constant Folding)	29
10.2	常量传播 (Constant Propagation)	29
10.3	死代码消除 (Dead Code Elimination)	29
10.4	公共子表达式消除 (CSE)	29
10.5	循环不变量外提 (Loop Invariant Code Motion)	29
10.6	内联 (Inlining)	30
10.7	强度削减 (Strength Reduction)	30
10.8	优化的局限性	30
10.9	这些优化和 MLOG 的关系	30
10.10	小结	31
11	回到 MLOG — 理论如何指导实践	32
11.1	MLOG 编译器和 x86 编译器的同构	32
11.2	MLOG 编译器的简化来源	32
11.3	共享的部分	32
11.4	如果给 MLOG 编译器加一个 x86 后端	32
11.5	MLOG 作为教学编译目标的优势	33
11.6	小结	33
12	到过程宏 — 编译时翻译的通用框架	34
12.1	过程宏本质上是编译器前端	34
12.2	和传统编译器的对比	34
12.3	适合放过程宏的场景	34
12.4	任何源语言 → 任何目标语言	35
12.5	过程宏的局限性	35
12.6	小结	35
13	翻译链的普遍性	36
13.1	三条链, 一个骨架	36
13.2	核心概念回顾	36
13.3	编译器设计的核心原则	36
13.4	推荐阅读	36
13.5	最后的话	36

1 序章：代码生成的普遍性

编译器是什么？答案很简单，但展开之后极其宏大：

> 编译器是一个**翻译器**——把一种语言的程序转换为另一种语言的等价程序。

每一层翻译丢掉一些抽象，添加一些细节，直到目标机能直接执行。

1.1 三条翻译链

把三条翻译链并排放在一起看：

层次	C → x86	Rust → LLVM	MLOG DSL → MLOG
1. 源语言	C 源码	Rust 源码	MLOG DSL
2. 中间层	GIMPLE / RTL	MIR → LLVM IR	AST → 三地址码 IR
3. 后端	x86 汇编	x86 / ARM / WASM	MLOG 指令
4. 最终	机器码	机器码	字符串文本

核心直觉

三条链的共同骨架：

源语言 → 中间表示 (IR) → 目标语言

中间的 IR 是编译器设计的**支点**。所有的分析、优化、变换都在 IR 上发生。前端的差异被 IR 吸收，后端的差异从 IR 出发。

1.2 每一层丢掉什么、添加什么

| 翻译步骤 | 丢掉 | 添加 | |:-|:-|:-| | C for → 汇编 | 循环语义的「为什么」 | 标签名、cmp 指令、jmp 指令 | | x = a + b * c → IR | 运算符优先级 | 临时变量 __t0、指令顺序 | | Rust match → LLVM | 模式匹配的穷尽性 | 跳转表/分支链 | | MLOG DSL while → MLOG | 块作用域 | 标签、条件跳转 |

编译器每往下走一层，就更接近机器的思维，更远离人的思维。

1.3 核心问题

本书围绕一个核心问题展开：

> 高级语言中的构造（变量、表达式、循环、函数、类型）在低层如何表示？

不局限于某一种语言或某一种目标——我们关注的是变换本身的**通用模式**。

1.4 本书结构

第一章：从 CPU 的视角看世界——它只能做什么 **第二至四章：**C → 汇编的经典三部曲——表达式、控制流、函数 **第五章：**寄存器分配——编译器的心脏 **第六章：**三地址码——编译器的通用语言 **第七章：**从 C 到 Rust——多了什么，少了什么 **第八章：**Rust 特有构造的编译——enum、trait、闭包 **第九章：**优化——编译器在帮你做什么 **第十章：**回到 MLOG——理论如何指导实践 **第十一章：**到过程宏——编译时翻译的通用框架 **第十二章：**翻译链的普遍性

每一章都从具体代码出发，展示它如何被一层层翻译，直到机器能理解的形式。

开始吧。

2 最低抽象 – CPU 看到的世界

 实验记录： 第一站：目标机的眼睛

写编译器的人必须理解目标机能做什么。取 x86-64 作为参考架构——不是为了精通汇编，而是为了理解任何 CPU 的底层共性。

2.1 寄存器：CPU 的零级缓存

寄存器是 CPU 内部最快的存储器。x86-64 有 16 个通用 64 位寄存器。

寄存器	常用角色	:- :-	rax	返回值、累加器		rbx	被调用者保存		rcx
	参数 4、计数器		rdx	参数 3、除法高 64 位		rsi	参数 2、源指针		rdi
	参数 1、目标指针		rsp	栈指针 – 指向栈顶		rbp	基址指针 – 栈帧底部 (可选)		
	r8-r15			参数 5-6、通用					

概念

寄存器的本质：CPU 能直接操作的最快存储。

操作寄存器的指令通常 1 个周期完成。操作内存的指令需要多周期（缓存命中/未命中差异极大）。

2.2 内存：大但慢

内存和寄存器之间通过 load（读入寄存器）和 store（写回内存）交互：

```
mov rax, [rbx]      ; load:  rax = *rbx (把 rbx 指向的地址的值读入 rax)
mov [rbx], rax      ; store: *rbx = rax
```

内存是字节数组——没有类型。32-bit 有符号整数和 32-bit 浮点数在内存里都是 4 个字节。区别只在你用什么指令操作它们。

2.3 核心指令

2.3.1 数据移动

```
mov rax, rbx        ; rax = rbx
mov rax, 42          ; rax = 42 (立即数)
mov rax, [rbx]       ; rax = 内存[rbx]
mov [rbx], rax       ; 内存[rbx] = rax
```

2.3.2 算术

```
add rax, rbx         ; rax = rax + rbx
sub rax, rbx         ; rax = rax - rbx
imul rax, rbx        ; rax = rax * rbx (有符号乘)
idiv rbx             ; rax = rdx:rax / rbx; rdx = 余数
neg rax              ; rax = -rax
and rax, rbx         ; rax = rax & rbx
or  rax, rbx         ; rax = rax | rbx
xor rax, rbx         ; rax = rax ^ rbx
shl rax, cl          ; rax = rax << cl
shr rax, cl          ; rax = rax >> cl (逻辑)
sar rax, cl          ; rax = rax >> cl (算术)
```

2.3.3 比较和跳转

```
cmp rax, rbx    ; 计算 rax - rbx, 设置标志位, 丢弃结果
je label        ; 相等则跳
jne label       ; 不等则跳
jl label        ; 小于 (有符号) 则跳
jg label        ; 大于 (有符号) 则跳
jle label       ; 小于等于
jge label       ; 大于等于
jmp label       ; 无条件跳转
```

概念

x86 的比较-跳转是两步操作:

1. `cmp a, b` 做减法, 只设置标志位 (ZF、SF、OF、CF), 不存储结果
2. `jxx` 根据标志位决定是否跳转

这和 MLOG 的 `jump label condition a b` 很不一样——MLOG 把比较和跳转合并成一条指令。但在三地址码层面, 它们等价。

2.3.4 函数调用

```
call func        ; push rip; jmp func
ret              ; pop rip (返回到调用者)
```

2.4 没有的东西

在汇编层面不存在的东西——编译器必须自己实现:

| 高层概念 | 汇编现实 | |:-|:-| | 变量名 | 寄存器编号或栈偏移 | | 类型 | 不同宽度的指令 | | 作用域 | 不存在——只有地址 | | 控制流结构 | `cmp + jmp` 组合 | | 函数 | `call + ret + 传参约定` | | 数组 | 基址 + 偏移 × 元素大小 | | 对象/struct | 基址 + 字段偏移 |

核心直觉

CPU 是一个极其简单的机器。

它能做的事: 从寄存器/内存读取数据, 做算术/位操作, 把结果写回寄存器/内存, 根据比较结果跳转。

它不知道什么是 `int`、什么是 `struct`、什么是 `for` 循环。它只知道: `mov`、`add`、`cmp`、`jmp`。

编译器的工作就是把 `int`、`struct`、`for` 循环全部翻译成 `mov`、`add`、`cmp`、`jmp`。

2.5 一个简单的 C → 汇编对照

C:

```
int add(int a, int b) {
    return a + b;
}
```

x86-64:

```
add:
    lea eax, [rdi + rsi]    ; eax = rdi + rsi (a + b)
    ret
```

C 看来很简单——两个参数加起来返回。汇编层面：

- a 在 rdi 中 (System V 调用约定, 第一个整数参数)
- b 在 rsi 中 (第二个参数)
- 结果放在 eax 中 (返回值)
- lea 是一条指令完成加法并存入目标 (不访问内存)

这就是编译的最底层——把语义映射到寄存器操作。理解了这一层, 往上走的每一层抽象都有根可循。

2.6 小结

- CPU 的核心操作: mov / 算术 / cmp+jump / call+ret
- 寄存器是最快的存储, 内存大但慢
- 高层概念 (类型、变量、循环) 在汇编层不存在
- 编译 = 把不存在的东西翻译成存在的东西
- x86 比较-跳转是两步; MLOG 的 jump 是一步——但三地址码层等价
- 这是所有编译器后端的共同祖先

3 C 表达式 → 汇编

 实验记录： 第二站：类型消失与三地址码

C 表达式是嵌套的树。汇编指令是扁平的序列。翻译的核心就是把树变成序列——这和 `rust2mlog` 中的 AST → 三地址码是同一个问题。

3.1 最简单的对照

C:

```
int x = a + b;
```

x86-64 (假设 `a` 在 `rsi`, `b` 在 `rdx`):

```
mov eax, esi      ; eax = a
add eax, edx      ; eax = eax + b
mov [x], eax      ; 存回内存
```

三条指令完成一条 C 语句。表达式内部已有隐含的三地址码结构: `temp = a; temp = temp + b; x = temp`。

3.2 复杂表达式分解

C:

```
int z = a + b * c - d / e;
```

编译器内部先把它变成三地址码:

```
t0 = b * c
t1 = a + t0
t2 = d / e
t3 = t1 - t2
z = t3
```

x86-64:

```
mov eax, [b]      ; t0 = b * c
imul eax, [c]
add eax, [a]      ; t1 = a + t0
mov ecx, [d]      ; t2 = d / e
cdq
idiv dword [e]
sub eax, ecx      ; t3 = t1 - t2 (wait, this is wrong...)
```

等等——这乱了。寄存器分配是一个独立的复杂问题。让我们用无限虚拟寄存器来简化思考。

3.3 用虚拟寄存器思考

 概念

无限虚拟寄存器模型: 假设有无限多个寄存器 `r0`, `r1`, `r2`, ...。

每个临时值占一个新寄存器。后面(第五章)再讨论如何把无限虚拟寄存器映射到有限的物理寄存器。

这个模型和三地址码几乎同构:


```
r3 = r1 op r2  ↔  op <opcode> r3 r1 r2
```

```
r0 = b * c
r1 = a + r0
r2 = d / e
r3 = r1 - r2
z  = r3
```

这三地址码和 MLOG 的 `op` 指令完全一致——只是寄存器名不同。

3.4 类型如何消失

C:

```
char c = 'A';      // 1 字节
short s = 1000;    // 2 字节
int i = 100000;    // 4 字节
long l = 100000;   // 8 字节
```

x86-64:

```
mov al, 65          ; char → 8-bit 寄存器
mov ax, 1000         ; short → 16-bit
mov eax, 100000      ; int → 32-bit
mov rax, 100000      ; long → 64-bit
```

类型信息在编译时驱动了**指令选择**——决定用 `mov al` (8 位) 还是 `mov eax` (32 位)。一旦指令生成, 类型就消失了。

3.5 指针和内存寻址

C:

```
int *p = &x;
int y = *p;      // y = *p
int z = p[3];    // z = *(p + 3)
```

汇编 (假设 `x` 在栈上偏移 `-4(rbp)`, `p` 在 `-8(rbp)`):

```
; int *p = &x;
lea rax, [rbp-4]    ; rax = &x
mov [rbp-8], rax    ; p = rax

; int y = *p;
mov rax, [rbp-8]    ; rax = p
mov eax, [rax]      ; eax = *p
mov [rbp-12], eax   ; y = eax

; int z = p[3]; = *(p + 3 * sizeof(int))
mov rax, [rbp-8]    ; rax = p
mov eax, [rax + 12] ; eax = *(p + 12) = p[3]
```

p[3] 在汇编中消失—变成 [rax + 12] (12 = 3 × sizeof(int) = 3 × 4)。

核心直觉

数组索引在编译时被**算术化**： arr[i] → *(arr + i * sizeof(element))

这完全是一个编译时计算—生成的汇编中只有加法和解引用。

3.6 结构体字段访问

C:

```
struct Point { int x; int y; };
struct Point p;
p.x = 10;
p.y = 20;
```

汇编:

```
; p 在栈上, 基址 rbp-8
mov dword [rbp-8], 10      ; p.x = 10 (offset +0)
mov dword [rbp-4], 20      ; p.y = 20 (offset +4)
```

p.x 和 p.y 在编译时被计算为栈偏移量。编译完成后没有“字段名”这个概念。

3.7 表达式翻译的通用模式

无论目标是 x86 还是 MLOG:

1. 把嵌套表达式树拍平为三地址码序列
2. 每个操作数可以是：立即数、虚拟寄存器、内存位置
3. 为每个子表达式结果分配临时存储（寄存器或临时变量）
4. 按依赖顺序排列指令

在 x86 编译器中，步骤 3-4 还需要寄存器分配（第五章）。在 MLOG 编译器中不需要—MLOG 变量无限制。

3.8 小结

- C 表达式是嵌套树，汇编是扁平序列
- 翻译 = 把树分解为三地址码
- 类型在编译时驱动指令选择，编译后消失
- 数组索引 arr[i] 变成 *(base + i * stride)
- 结构体字段变成编译时的偏移量
- 三地址码是所有编译目标的共同语言
- x86 有寄存器压力，MLOG 没有—这是关键差异

4 C 控制流 → 汇编

 实验记录：第三站：cmp + jmp = 一切

C 的 if、while、for、switch 在汇编层全部归结为 cmp + 条件跳转 + 无条件跳转。这和 MLOG 的 jump 模式完全相同。

4.1 if-else

C:

```
if (x > 0) {  
    y = 1;  
} else {  
    y = -1;  
}
```

汇编（伪代码，虚拟寄存器）:

```
cmp r0, 0          ; x > 0 ?  
jle __else         ; if x <= 0, goto else  
mov r1, 1          ; y = 1  
jmp __end  
__else:  
mov r1, -1         ; y = -1  
__end:
```

三地址码 IR:

```
op gt __t0 x 0  
jump __else equal __t0 false ; x <= 0  
set y 1  
jump __end  
:__else  
set y -1  
:__end
```

核心直觉

if-else 的编译模式和 MLOG 完全一致:

1. 计算条件 → 临时变量
2. 条件为假 → 跳转到 else (或跳过 then)
3. 执行 then 分支
4. 无条件跳到 end (如果有 else)
5. else 分支
6. end 标签

x86 用 cmp + jle, MLOG 用 op gt + jump equal false——语义等价，只是操作码不同。

4.2 while 循环

C:

```
while (n > 0) {
    sum += n;
    n--;
}
```

汇编:

```
__loop:
    cmp r0, 0          ; n > 0 ?
    jle __end          ; if n <= 0, exit loop
    add r1, r0          ; sum += n
    sub r0, 1          ; n--
    jmp __loop
__end:
```

三地址码 IR:

```
:__loop
    op gt __t0 n 0
    jump __end equal __t0 false    ; n <= 0
    op add sum sum n
    op sub n n 1
    jump __loop
:__end
```

x86 的 while 和 MLOG 的 while 在三地址码层面**完全相同**——都是标签 + 条件跳转 + 回跳的组合。

4.3 for 循环

C 的 for 是 while 的语法糖:

```
for (int i = 0; i < 10; i++) {
    sum += i;
}
```

等价于:

```
int i = 0;
while (i < 10) {
    sum += i;
    i++;
}
```

编译器没有“for 循环”这个概念——for 在解析阶段就被展开成 while 的形式，然后按 while 的模式生成代码。

4.4 switch

C:

```
switch (x) {
    case 0: y = 10; break;
```

```

    case 1: y = 20; break;
    case 2: y = 30; break;
    default: y = 0; break;
}

```

有两种编译策略：

4.4.1 策略一：链式比较（case 少时）

```

    cmp x, 0; jne __case1
    mov y, 10; jmp __end
__case1:
    cmp x, 1; jne __case2
    mov y, 20; jmp __end
__case2:
    cmp x, 2; jne __default
    mov y, 30; jmp __end
__default:
    mov y, 0
__end:

```

$O(n)$ ——和 if-else 链没区别。

4.4.2 策略二：跳转表（case 密集时）

```

    mov rax, x
    cmp rax, 2          ; 边界检查
    ja __default
    jmp [jt + rax*8]     ; 间接跳转
jt: .quad __case0, __case1, __case2

```

$O(1)$ ——用 case 值做索引直接跳转。



概念

跳转表是一个编译时常量数组，存储每个 case 的目标地址。

jmp [jt + rax*8] 用 case 值做数组索引，一次跳转完成分发。

这是 switch 比 if-else 链快的根本原因。

4.5 短路求值

```

if (a != 0 && b / a > 10) { ... }

```

&& 的短路语义：如果 $a == 0$ ，不能执行 b / a （会除零）。

汇编：

```

    cmp a, 0
    je __skip          ; a == 0 → 整个条件为 false
    mov eax, b
    cdq
    idiv a              ; b / a
    cmp eax, 10

```

```
    jle __skip          ; b/a <= 10 → 整个条件为 false
    ; ... then body ...
__skip:
```

🔍 核心直觉

短路求值 = 提前跳转。

&&: 第一个操作数为假 → 跳过整个条件, 直接到 `false` 分支 ||: 第一个操作数为真 → 跳过整个条件, 直接到 `true` 分支

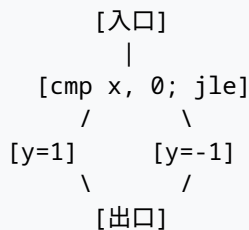
这和 MLOG 的短路求值逻辑完全一致。

4.6 控制流图 (CFG)

定义

控制流图 (Control Flow Graph) 是把指令序列划分为**基本块**, 用箭头连接跳转关系。

- 基本块: 连续执行的指令序列, 入口只能是最顶部, 出口只能是最底部
- 边: 从 `jump` 指令指向目标标签



CFG 是编译器分析和优化的基础数据结构。数据流分析、死代码消除、循环检测——全部建立在 CFG 上。

4.7 小结

- `if/while/for` 全部归结为 `cmp` + 条件跳转 + 无条件跳转
- `for` 是 `while` 的语法糖——展开后没有区别
- `switch` 有两种编译策略: 链式比较 vs 跳转表
- 短路求值 = 提前跳转 (&& 遇假则跳, || 遇真则跳)
- 控制流图是编译优化的基础
- x86 和 MLOG 的控制流编译模式**完全相同**——只是指令名不同

5 C 函数 → 汇编

🔧 实验记录：第四站：栈帧与调用约定

函数调用是编译中最复杂的部分——涉及参数传递、栈管理、返回值、寄存器保存和恢复。

5.1 调用约定的意义

定义

调用约定 (calling convention) 是调用者和被调用者之间关于参数传递、栈清理和寄存器使用的协议。

没有它，调用者不知道把参数放哪里，被调用者不知道返回值放哪里，两边的寄存器使用互相覆盖。

不同的 ABI (Application Binary Interface) 定义了不同的调用约定。

5.1.1 System V AMD64 ABI (Linux/macOS 上 C 的默认约定)

方面	规则	整数参数 1-6	浮点参数 1-8
:- :-	rdi rsi rdx rcx r8 r9	xmm0-xmm7	
多余参数	栈传递 (从右到左压栈)	返回值	rax (整数) 或 xmm0 (浮点)
被调用者保存	rbx rbp r12-r15 - 被调用者必须恢复	调用者保存	rax rcx rdx rsi rdi r8-r11 - 调用者自己保存
栈对齐	调用前 16 字节对齐		



概念

被调用者保存 vs 调用者保存：

- **被调用者保存**：函数如果要用这些寄存器，必须先 `push` 旧值，返回前 `pop` 回去。
- **调用者保存**：如果调用者在这些寄存器中存了重要数据，必须在调用前自己保存。

这避免了每个 `call` 都要保存所有寄存器——只有真正需要的才保存。

5.2 栈帧

```
int add(int a, int b) {  
    int result = a + b;  
    return result;  
}
```

这个函数的栈帧 (简化)：

高地址	
+-----+	
返回地址	← <code>call</code> 指令自动 <code>push</code>
+-----+	
旧 <code>rbp</code>	← <code>push rbp; mov rbp, rsp</code>
+-----+	← <code>rbp</code>
<code>result (-4)</code>	← 局部变量
+-----+	← <code>rsp</code>
低地址	

汇编：

```
add:  
    push rbp  
    mov rbp, rsp  
    sub rsp, 16           ; 分配局部变量空间 (这里只需要 4, 对齐到 16)
```

```

mov [rbp-4], edi    ; result = a (a 在 edi 中)
add [rbp-4], esi    ; result += b (b 在 esi 中)
mov eax, [rbp-4]    ; 返回值 = result
leave              ; mov rsp, rbp; pop rbp
ret

```

🔍 核心直觉

栈帧就是函数在栈上划出的一小块私有空间。

里面放着：

- 旧的 `rbp`（链式回溯）
- 返回地址（知道返回哪里）
- 局部变量
- 溢出的寄存器
- 传给下一级函数的额外参数

5.3 `call` 和 `ret` 的内部操作

```
call func
```

等价于：

```

push rip+5    ; 把下一条指令地址压栈
jmp func      ; 跳到函数

```

```
ret
```

等价于：

```
pop rip    ; 从栈顶弹出返回地址，跳到那里
```

整个机制依赖栈——`call` 把返回地址 `push` 到栈上，`ret` 从栈上 `pop` 回来。递归就是这个机制的自动应用。

5.4 递归的栈展开

```

int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

```

调用 `factorial(3)` 时栈的样子：

```

factorial(1)  rbp → [n=1, ...]
factorial(2)  rbp → [n=2, ...]
factorial(3)  rbp → [n=3, ...]
main         rbp → [...]

```

每次递归调用在栈上 `push` 一个新的帧，返回时 `pop` 掉。如果递归太深——`factorial(1000000)`——栈溢出。

5.5 尾调用优化

```
int tail_sum(int n, int acc) {
    if (n == 0) return acc;
    return tail_sum(n - 1, acc + n); // 尾调用
}
```

尾调用的特点：调用者的栈帧在调用之后不再需要。编译器可以复用调用者的栈帧：

```
tail_sum:
    cmp edi, 0
    jne .recurse
    mov eax, esi        ; return acc
    ret
.recurse:
    add esi, edi        ; acc += n
    sub edi, 1          ; n--
    jmp tail_sum        ; 不是 call! 是 jmp—复用栈帧
```

核心直觉

尾调用优化把 `call + ret` 变成 `jmp`：

正常调用：`call func; ...; ret` — 每个调用占用一个新栈帧 尾调用：`jmp func` — 复用当前栈帧， $O(1)$ 栈空间

这就是为什么尾递归不会栈溢出。

5.6 和 MLOG 的对比

特性	x86 函数	MLOG	:- :- :-	调用机制	call / ret + 栈	不支持
参数传递	寄存器 + 栈	N/A		局部变量	栈帧	全局变量（唯一名称化）
自动（栈）	不可能			编译策略	复杂的栈帧管理	内联展开所有逻辑

MLOG 没有调用栈——这不是 bug，是设计约束。我们的 MLOG 编译器通过内联展开模拟“函数”——把所有逻辑铺平成一条指令流。

概念

函数编译的通用模式：

1. 调用者把参数放到约定位置（寄存器或栈）
2. `call` 保存返回地址
3. 被调用者建立栈帧（`push rbp; mov rbp, rsp`）
4. 执行函数体
5. 把返回值放入约定寄存器
6. 恢复栈帧，`ret` 返回

5.7 小结

- 调用约定是调用者和被调用者的协议
- 栈帧 = 函数在栈上的私有空间
- `call` = `push` 返回地址 + `jmp`; `ret` = `pop` + `jmp`
- 递归依赖栈，尾调用可以优化为循环
- MLOG 没调用栈——所有逻辑必须内联展开
- 函数编译的核心问题：参数传递、栈管理、寄存器保存

6 寄存器分配 — 编译器的心脏

 实验记录： 第五站：有限资源的优雅分配

在三地址码中，我们假设有无穷多个虚拟寄存器。但真实的 CPU 只有十几个通用寄存器。

寄存器分配就是把无限虚拟寄存器映射到有限物理寄存器的过程。这是编译器中公认最难、也最重要的优化步骤之一。

6.1 问题的核心

```
; 三地址码（无限虚拟寄存器）
r0 = a + b
r1 = r0 * c
r2 = d - e
r3 = r1 + r2
return r3
```

只有 4 个物理寄存器怎么办？如果虚拟寄存器超过物理寄存器数量，就必须把一些值**溢出**（spill）到内存（栈）中，需要时再 load 回来。

```
; 假设只有 2 个寄存器：rA, rB
load rA, [a]
load rB, [b]
add rA, rB           ; rA = r0 = a + b
store [r0_tmp], rA   ; 溢出 r0 到栈
load rA, [c]
mul rA, [r0_tmp]     ; rA = r1 = r0 * c
store [r1_tmp], rA   ; 溢出 r1
load rA, [d]
load rB, [e]
sub rA, rB           ; rA = r2 = d - e
load rB, [r1_tmp]    ; 恢复 r1
add rA, rB           ; rA = r3 = r1 + r2
```

因为寄存器不够，额外产生了 4 条 load/store 指令——纯开销。

6.2 活跃范围 (Live Range)

定义

一个虚拟寄存器的**活跃范围**是从它被定义（写入）到最后一次被使用之间的所有指令。

```
r0 = a + b           ← r0 定义
r1 = r0 * c          ← r0 最后使用（之后 r0 dead）
r2 = d - e
r3 = r1 + r2
```

r0 的活跃范围：从第 1 行到第 2 行。 r1 的活跃范围：从第 2 行到第 4 行。 r2 的活跃范围：从第 3 行到第 4 行。

两个活跃范围不重叠的虚拟寄存器可以共享同一个物理寄存器。

核心直觉

寄存器分配的核心思想：

如果两个变量永远不会“同时存活”，它们就可以复用同一个物理寄存器。
就像停车位——如果两辆车从不同时在场，它们可以共用一个车位。

6.3 图着色 (Graph Coloring)

寄存器分配的经典方法——把问题建模为干涉图 (interference graph) 的着色。

1. 每个虚拟寄存器是图中的一个节点
2. 如果两个虚拟寄存器的活跃范围重叠，它们之间有一条边
3. 用 K 种颜色 ($= K$ 个物理寄存器) 给图着色
4. 相邻节点不能同色
5. 如果无法着色，选择某个节点溢出到内存

```
; 干涉图示例
r0 - r1      (r0 和 r1 同时存活)
r1 - r2      (r1 和 r2 同时存活)
r3 独立      (r3 和其他都不重叠)
```

如果只有 2 个寄存器: $r0$ 和 $r2$ 可以共享 (它们没有边), $r1$ 需要一个单独的, $r3$ 可以和 $r0/r2$ 共享。

概念

图着色寄存器分配 = NP 完全问题。

编译器用启发式方法近似求解——贪心着色 + 必要时溢出。

LLVM 使用贪心图着色 + 线性扫描的混合策略。GCC 使用基于优先级的图着色。

6.4 线性扫描分配

更简单的启发式方法——按指令顺序扫描，维护一个活跃列表：

1. 遇到新的虚拟寄存器 → 分配一个空闲物理寄存器
2. 如果物理寄存器用完 → 选择活跃范围最远的溢出
3. 虚拟寄存器死亡 → 释放物理寄存器

线性扫描比图着色快得多 ($O(n)$ vs NP)，生成的代码稍差 (约 5-10% 性能损失)，但在 JIT 编译器 (如 V8) 中广泛使用。

6.5 这和 MLOG 有什么关系

MLOG 没有物理寄存器限制。每个 MLOG 变量相当于一个虚拟寄存器——处理器内部管理它们的存储。

核心直觉

MLOG 编译器不需要寄存器分配——这是它和 x86 编译器最大的架构差异。

这也是为什么 MLOG 编译器只有 1000 行——它绕过了编译器中最难的一个问题。

但理解寄存器分配仍然是必要的：它让你理解 x86/Rust 编译器在做什么，以及为什么 MLOG 编译器的设计可以如此简洁。

6.6 小结

- 寄存器分配 = 无限虚拟 → 有限物理的映射
- 活跃范围决定哪些变量可以共享寄存器
- 图着色 = 干涉图 + 着色 + 必要时溢出

- 线性扫描 = 更快、稍差
- MLOG 不需要寄存器分配—变量数量无限制
- 这就是 MLOG 编译器和 x86 编译器复杂度差异的根源

7 三地址码 — 编译器的通用语言

✍ 实验记录：第六站：IR 的深层理论

三地址码已经出现了多次——在第二章（C 表达式分解）、在 rust2mlog 的第五章（IR 设计）、在 MLOG 本身的指令格式中。

这一章把三地址码作为独立的理论对象来审视：它为什么是编译器的“通用语言”？

7.1 三地址码的形式

定义

三地址码是一种中间表示，每条指令最多有三个操作数：

```
result = operand1 <operator> operand2
```

或者单操作数形式：

```
result = <operator> operand
```

操作数可以是：常量、变量、临时变量。

四种基本指令类型：

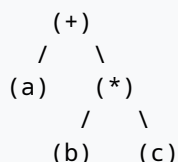
类型	形式	示例
二元运算	$x = y \text{ op } z$	$t1 = a + b$
一元运算	$x = \text{op } y$	$t2 = -t1$
复制	$x = y$	$t3 = b$
条件跳转	$\text{if } x \text{ op } y \text{ goto } L$	$\text{if } t1 < 10 \text{ goto loop}$
无条件跳转	$\text{goto } L$	goto end

这恰好是 MLOG 的 `op + set + jump` 指令族——MLOG 天然就是三地址码。

7.2 为什么三地址码是“最佳”IR

7.2.1 对比：语法树

语法树（AST）保留了源代码的嵌套结构：



这很漂亮，但不适合做分析和优化——指令顺序不明确，操作数位置不统一。

7.2.2 对比：二地址码

x86 汇编很多指令是二地址的：

```
add rax, rbx ; rax = rax + rbx (破坏了 rbx 还是 rax? 是 rax)
```

二地址码的问题是：源操作数被破坏，需要额外指令来保留值。

7.2.3 三地址码的黄金平衡

- 每条指令只做一件事（不像语法树可以有任意深度）
- 不破坏操作数（不像二地址码）
- 指令顺序明确（优化器可以重新排列）
- 操作数位置固定（便于写优化 pass）

🔍 核心直觉

“三”不是随便选的。

- 一地址码（累加器模型）：太受限，需要大量 load/store
- 二地址码：破坏性操作，不够灵活
- 三地址码：刚好表达了“取两个值，算一下，放结果”
- 四地址码：多余（大多数运算只需要两个输入和一个输出）

三是编译优化的甜点——足够表达所有操作，但不冗余。

7.3 SSA 形式：更严格的三地址码

定义

静态单赋值（Static Single Assignment, SSA）要求每个变量在程序中只被赋值一次。

```
// 非 SSA
x = a + b
x = x + 1
y = x * 2

// SSA
x1 = a + b
x2 = x1 + 1
y = x2 * 2
```

如果有多个控制流路径都定义同一个变量，用 **phi 函数** 合并：

```
if cond:
    x1 = 1
else:
    x2 = 2
x3 = phi(x1, x2)    // 根据来自哪个分支选择值
```

SSA 的主要优势：每个变量只有一个定义点——使用 - 定义链变得平凡。很多优化（常量传播、死代码消除）在 SSA 形式上更容易实现。

LLVM IR 就是 SSA 形式的三地址码。

7.4 MLOG 和 x86 在三地址码下的统一

| | x86 | MLOG | | :-| :-| :-| | 二元运算 | add rax, rbx (二地址) | op add result a b (三地址) | | 赋值 | mov rax, 5 | set result 5 | | 条件跳转 | cmp a, b; je label (两步) | jump label equal a b (一步) |

从三地址码视角看，x86 的二地址指令只是语法差异——每条 x86 指令可以规范化为三地址形式。MLOG 本身就是三地址码。

🏰 概念

计算 IR 的终极价值：

不管你编译到 x86、ARM、MLOG 还是 WebAssembly——三地址码 IR 提供了统一的抽象。

- 前端：语言特定 → IR
- IR：分析和优化
- 后端：IR → 目标语言

这就是为什么 LLVM 能支持几十种前端语言和几十种后端目标——因为它们都在 LLVM IR (SSA 形式的三地址码) 上汇合。

7.5 小结

- 三地址码是编译器的通用中间语言
- 语法树太深，二地址太受限，三地址刚好
- SSA = 每个变量只赋值一次 + phi 函数合并
- MLOG 的 op/set/jump 是天然的三地址码
- LLVM IR 是 SSA 形式的三地址码
- IR 是编译器架构的支点——前后端解耦

8 从 C 到 Rust — 多了什么

 实验记录：第七站：新语言的编译挑战

前六章讨论的是 C → 汇编的经典模式。现在转到 Rust——它在编译层面比 C 多了什么？

答案出人意料：大部分“多的东西”在编译时解决，运行时和 C 几乎一样。

8.1 C 编译器已经解决的问题

| 问题 | C 的方式 | Rust 也继承 | |:-|:-|:-| | 表达式分解 | 三地址码 | 相同 | | 控制流翻译 | cmp + jmp | 相同 | | 函数调用 | 调用约定 + 栈帧 | 相同 | | 类型擦除 | 编译后消失 | 相同 | | 结构体布局 | 字段偏移计算 | 相同 | | 寄存器分配 | 图着色 | 相同 |

Rust 的编译器后端 (LLVM) 和 C/C++ 共享——生成的机器码在结构上和 C 编译器生成的没有本质区别。

8.2 Rust 多了什么——在编译时

8.2.1 所有权和借用

Rust 最独特的特性——所有权、借用、生命周期——全部在编译时检查。

```
Rust: let y = &x; // 借用检查：y 的存活范围不能超过 x
C:    int *y = &x; // 没有任何检查
```

编译完成后，&x 和 C 的 &x 生成相同的汇编——都是 `lea rax, [rbp-4]`。所有权的信息完全消失了。

核心直觉

Rust 的所有权系统是编译时验证，运行时零成本。

这适用于：所有权、借用、生命周期、Send/Sync、Unpin.....

编译时验证 → 编译后擦除 → 运行时性能和 C 完全一样。

8.2.2 泛型和单态化 (Monomorphization)

```
fn max<T: Ord>(a: T, b: T) -> T {
    if a > b { a } else { b }
}

let x = max(3, 5);           // T = i32
let y = max(3.0, 5.0);       // T = f64
```

编译器生成两份独立的机器码——一份给 i32，一份给 f64。和手动写两个 `max_i32`、`max_f64` 函数完全等价。

```
max_i32:                                max_f64:
    cmp edi, esi                        ucomisd xmm0, xmm1
    jge .a                              jae .a
    mov eax, esi                        movapd xmm0, xmm1
    ret                                .a:
.a:                                    ret
    mov eax, edi
    ret
```

单态化的代价是**代码膨胀**——每个类型组合生成一份代码。但运行时性能和手写特化代码一样快。

8.3 动态分发: trait object

```
fn draw(shape: &dyn Drawable) {  
    shape.draw();  
}
```

编译器不能为 `dyn Drawable` 做单态化——因为不知道运行时具体是哪个类型。改用 **vtable**:

```
; shape 在 rdi 中  
; &dyn Drawable = (data_ptr, vtable_ptr) - fat pointer  
mov rax, [rdi + 8]      ; rax = vtable_ptr  
call [rax + 0]          ; 调用 vtable[0] = draw 方法
```

概念

Fat Pointer: `&dyn Trait` 是 16 字节——8 字节数据指针 + 8 字节 vtable 指针。
vtable 是一个函数指针数组，每个 trait 方法占一个槽位。

C++ 的虚函数和 Rust 的 trait object 使用完全相同的机制。

8.4 Rust 仍然面临和 C 相同的编译问题

尽管 Rust 有更丰富的类型系统，编译后端面对的问题没有变:

- 表达式仍然需要分解为三地址码
- 控制流仍然是 `cmp + jmp`
- 函数仍然是栈帧 + 调用约定
- 寄存器仍然有限，仍然需要分配
- 优化 pass 仍然在做常量折叠、死代码消除、循环变换

Rust 的创新在**前端**（类型检查、借用验证），不在后端。后端的挑战和 C 编译器完全一样。

8.5 小结

- Rust 编译后端 (LLVM) 和 C/C++ 共享——底层相同
- 所有权/借用/生命周期 = 编译时验证，编译后擦除
- 泛型 = 编译时复制粘贴（单态化）
- trait object = vtable + fat pointer (和 C++ 虚函数相同)
- 编译后端的核心挑战（表达式、控制流、函数、寄存器）没变
- Rust 的创新在前端，不在后端

9 Rust 特有构造 → 汇编

🔧 实验记录： 第八站：enum / match / 闭包 / async

Rust 有几个 C 没有的语言构造——enum（带数据的 tagged union）、match 穷尽模式匹配、闭包、async/await。它们在汇编层怎么表示？

9.1 enum 和 match

```
enum Option<T> {  
    None,  
    Some(T),  
}  
  
fn unwrap_or_default(x: Option<i32>) -> i32 {  
    match x {  
        Some(v) => v,  
        None => 0,  
    }  
}
```

汇编层实现是 **tagged union**：一个 tag（判别符）+ union（数据）。

```
; Option<i32> 的内存布局 (niche 优化前)  
; offset 0: tag (0 = None, 1 = Some)  
; offset 4: data (在 Some 时有效, 4 字节对齐后可能偏移 8)
```

match 编译为：

```
    cmp [rdi], 0           ; tag == None ?  
    je .none  
    mov eax, [rdi + 4]     ; Some(v) → 取 v  
    ret  
.none:  
    xor eax, eax           ; return 0  
    ret
```

9.1.1 Niche 优化

Option<T> 利用了一个事实：引用不可能为 null。所以 None = null pointer：

```
Option<T> 的内存布局：  
Some(ptr) → 存储 ptr  
None      → 存储 0x0 (null)
```

不需要额外的 tag 字段——嵌入在指针值的 null 状态中。

🔍 核心直觉

Option<Box<T>>、Option<T>、Option<NonZeroU32> 都享受 niche 优化。

编译器找到一个类型的“不可能值”，用那个值表示 None。这完全免费——不需要额外的 tag 字节。

9.1.2 跳转表

当 match 的 arm 很多且判别符的值密集时，用跳转表（和 C switch 一样）：

```
match x {
  0 => ..., 1 => ..., 2 => ..., ..., 10 => ...
}
```

```
cmp eax, 10; ja .default
jmp [jt + rax*8]
jt: .quad .L0, .L1, .L2, ..., .L10
```

9.2 闭包

9.2.1 无捕获闭包

```
let f = |x| x + 1;
```

不捕获任何环境变量 → 就是普通函数指针：

```
// 编译后等价于
fn __closure_1(x: i32) -> i32 { x + 1 }
let f = __closure_1 as fn(i32) -> i32;
```

9.2.2 捕获闭包

```
let base = 10;
let f = |x| x + base;
```

捕获了 base → 变成一个 struct + call 方法：

```
// 编译器生成的
struct __Closure_1 { base: i32 }

impl Fn(i32) -> i32 for __Closure_1 {
  fn call(&self, x: i32) -> i32 { x + self.base }
}

let f = __Closure_1 { base: 10 };
```

汇编层面：f(x) 变成加载 f.base + 做加法的普通指令序列。没有动态分发。

9.2.3 trait object 闭包

```
let f: Box<dyn Fn(i32) -> i32> = Box::new(|x| x + base);
```

dyn Fn → vtable。调用 f(x) → load vtable → call vtable[0]。

9.3 async / .await

```
async fn fetch() -> i32 { ... }
```

编译器把 async 函数转换为状态机：

```
// 编译器生成的
enum FetchFuture {
    Start,
    AfterFirstAwait { /* 保存的局部变量 */ },
    Done,
}

impl Future for FetchFuture {
    fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<i32> {
        match *self {
            FetchFuture::Start => {
                // ...启动异步操作...
                *self = FetchFuture::AfterFirstAwait { ... };
                Poll::Pending
            }
            FetchFuture::AfterFirstAwait { ... } => {
                // ...继续...
                Poll::Ready(result)
            }
            FetchFuture::Done => panic!("polled after ready"),
        }
    }
}
```

概念

async fn 的核心变换：函数体 → 状态机。

每个 .await 点是状态机的一个状态。局部变量跨 .await 存活 → 保存在状态机结构体中。
这是 Rust 编译器中最复杂的变换之一——但输出仍然是普通的汇编。

9.4 所有 Rust 构造的汇编归宿

| Rust 构造 | 汇编表示 | |:-|:-| | enum | tag + union | | match | cmp + jmp / 跳转表 | | Option<T> | niche 优化后可能是 null | | 闭包 (无捕获) | 函数指针 | | 闭包 (捕获) | struct + 函数调用 | | trait object | fat pointer + vtable | | async fn | 状态机 enum | | .await | 状态机的 poll 方法 | | Box<T> | 指针 (和 C malloc 一样) | | &T | 指针 | | Vec<T> | ptr + len + cap (3 个字段) |

所有高级抽象最终都变成：mov、add、cmp、jmp、call、ret。

核心直觉

这是 Rust 设计的精妙之处：丰富的类型系统 + 零成本抽象。

编译器在编译时做了大量的分析和变换——但最终生成的机器码和精心手写的 C 一样精简。

9.5 小结

- enum = tagged union (tag + union 的内存布局)
- match = cmp + jmp 或跳转表
- Niche 优化让 Option<&T> 等零额外开销
- 闭包 = 函数指针 (无捕获) 或 struct (捕获)
- async fn = 状态机变换
- 所有高级构造最终变成 mov/add/cmp/jmp/call/ret
- 编译后的 Rust 和编译后的 C 在汇编层几乎无法区分

10 优化 — 编译器在帮你做什么

 实验记录： 第九站：不改变语义，只改善性能

编译器不只做翻译——它还做优化。优化的原则是在不改变可观察行为的前提下，让代码更快或更小。

10.1 常量折叠 (Constant Folding)

```
int x = 2 + 3 * 4;
```

编译器在编译时计算：

```
int x = 14;    // 编译后
```

不仅是算术——字符串长度、数组大小、简单的条件判断都可能被折叠。

10.2 常量传播 (Constant Propagation)

```
int a = 5;
int b = a + 3;    // → 8
int c = b * 2;    // → 16
```

经过传播和折叠：

```
int a = 5;
int b = 8;
int c = 16;
```

如果后续没人用 `a` 和 `b`，它们会被死代码消除干掉，最后只剩 `int c = 16;`。

10.3 死代码消除 (Dead Code Elimination)

```
int x = expensive_calculation();
return 42;
```

`x` 被定义但从未使用 → `expensive_calculation()` 可以被删除。

10.4 公共子表达式消除 (CSE)

```
int x = a * b + c;
int y = a * b + d;    // a * b 重复计算
```

优化后：

```
int tmp = a * b;
int x = tmp + c;
int y = tmp + d;
```

10.5 循环不变量外提 (Loop Invariant Code Motion)

```
for (int i = 0; i < n; i++) {
    arr[i] = a * b + c;    // a * b + c 在每次迭代中相同
}
```

优化后：

```
int tmp = a * b + c;
for (int i = 0; i < n; i++) {
    arr[i] = tmp;
}
```

10.6 内联 (Inlining)

```
static int add(int a, int b) { return a + b; }
int x = add(3, 4);
```

内联后：

```
int x = 3 + 4;    // 然后常量折叠 → 7
```

内联不仅省了 `call/ret` 的开销，更重要的是暴露了更多的优化机会——常量传播、CSE 等都能跨函数执行。

10.7 强度削减 (Strength Reduction)

```
; 乘法 → 移位 + 加法
imul rax, 8    →    lea rax, [rax*8]

; 除法 → 乘法 + 移位 (编译器用魔法常量)
idiv rcx      →    mov rax, magic; mul; shr...
```

🔍 核心直觉

优化的哲学：

程序员写 **语义清晰** 的代码。编译器生成 **性能最优** 的代码。

$a * 8$ 和 $a \ll 3$ 语义相同，但后者更快。程序员应该写 $a * 8$ （它表达了“乘以 8”的意图），编译器负责把它变成移位。

这是分工：人关注可读性，编译器关注性能。

10.8 优化的局限性

编译器不能优化一切：

- 不能改变 I/O 的顺序
- 不能删除 `volatile` 访问
- 不能跨编译单元做激进优化（除非 LTO）
- 不能确定指针别名时不能重排内存访问
- 不能展开递归深度不确定的函数

编译优化是**保守的**——它必须在保证正确性的前提下做变换。

10.9 这些优化和 MLOG 的关系

rust2mlog 中的 MLOG 编译器目前几乎是**零优化**的——直接从 $AST \rightarrow IR \rightarrow MLOG$ 。

但所有这些优化技术都可以应用到 MLOG 编译器上：

- CSE 可以减少 MLOG 中的冗余 `op` 指令
- 常量折叠可以在编译时计算常量表达式

- 死代码消除可以删除无用的 `set` 指令
- 循环不变量外提可以减少 MLOG 循环体中的指令数

MLOG 优化器和 x86 优化器使用的是同一种理论——只需要适配指令集。

10.10 小结

- 优化的原则：不改变语义，改善性能
- 常量折叠/传播、死代码消除、CSE、循环不变量外提、内联、强度削减
- 内联是最重要的优化——它暴露其他所有优化机会
- 编译器优化是保守的——安全第一
- MLOG 编译器也可以应用相同的优化 `pass`
- 编译优化的理论是通用的——跨越目标架构

11 回到 MLOG — 理论如何指导实践

🔗 实验记录：第十站：用前九章的理论重新审视 rust2mlog

前九章建立了从 C 到 Rust 到 x86 的完整编译理论。现在回到 rust2mlog——用这套理论解释 MLOG 编译器的工作。

11.1 MLOG 编译器和 x86 编译器的同构

把两者放在同一张表里对照：

阶段	x86 编译器	MLOG 编译器	:- :- :-	解析	C 语法 → AST DSL 语法 → AST
中间表示	AST → GIMPLE/LLVM IR	AST → 三地址码 IR	表达式	树 → 三地址码 + 寄存器分配	树 → 三地址码 (不需寄存器分配)
控制流	if/while/for → cmp+jump	if/while/loop → jump	函数	调用约定 + 栈帧	不支持 (必须内联展开)
变量	虚拟寄存器 → 物理寄存器 + 溢出	全局变量 (唯一名称化)			

🔍 核心直觉

MLOG 编译器和 x86 编译器是同一棵树上结的不同果实。

三地址码是它们共同的根。差异在于叶子——目标指令集的不同约束。

x86：有限寄存器、有调用栈、有内存层次结构 MLOG：无限变量、无调用栈、无内存层次结构
编译器前端可以完全共享，后端各自适配。

11.2 MLOG 编译器的简化来源

MLOG 编译器比 x86 编译器简单得多。原因：

1. 不需要寄存器分配 — MLOG 的变量模型不需要映射到物理寄存器
2. 不需要栈帧管理 — 没有 call/ret, 没有 rbp/rsp 操作
3. 不需要指令选择 — MLOG 的 op 指令就是三地址码的直接对应
4. 不需要指令调度 — MLOG 没有流水线, 指令顺序不重要

这四条中的每一条在 x86 编译器中都是一个几千行的 pass。

11.3 共享的部分

尽管后端不同，核心前端逻辑完全相同：

```
// 这个函数在 x86 编译器和 MLOG 编译器中的逻辑是一样的
fn lower_binary_expr(lhs: &Expr, op: BinaryOp, rhs: &Expr) -> IrValue {
    let lhs_val = lower_expr(lhs); // 递归处理左子树
    let rhs_val = lower_expr(rhs); // 递归处理右子树
    let result = new_temp();        // 分配临时变量
    emit(IrInstr::Op {              // 生成三地址码
        opcode: map_op(op),
        result: result.clone(),
        lhs: lhs_val,
        rhs: rhs_val,
    });
    IrValue::Var(result)
}
```

表达式分解、控制流标签生成、变量名管理——这些在 x86 和 MLOG 之间是可复用的代码。

11.4 如果给 MLOG 编译器加一个 x86 后端

核心文件结构：


```
mlog-compiler/
├─ parser.rs      # DSL → AST (前后端共享)
├─ ir.rs          # AST → 三地址码 IR (前后端共享)
├─ codegen_mlog.rs # IR → MLOG 文本
└─ codegen_x86.rs # IR → x86 汇编 (新增)
```

只需要加一个文件——因为 IR 是前端和后端之间的稳定接口。

概念

这就是 LLVM 架构的精髓：

```
前端 (C/Rust/Swift/...) → LLVM IR ← 后端 (x86/ARM/WASM/...)
```

三地址码 IR 是所有这些编译器的共同语言。

MLOG 编译器也遵循相同的架构——只是目标不是机器码，而是 MLOG 文本。

11.5 MLOG 作为教学编译目标的优势

MLOG 作为编译目标有几个独特的教学优势：

特性	教学价值	:- :-	天然三地址码	不需要指令选择阶段	无限变量	不需要
寄存器分配	无调用栈	简化函数编译 (内联)	人类可读	输出可以手动检查和调试		
指令集小	完整的后端不到 200 行					

这就是为什么用 MLOG 来学习编译器设计比用 x86 容易得多——它去掉了最复杂的两块（寄存器分配和指令选择），同时保留了编译器的核心骨架。

11.6 小结

- MLOG 和 x86 编译器共享相同的 IR 层
- 差异在后端：MLOG 不需要寄存器分配、栈帧、指令选择
- 前端逻辑（解析、三地址码生成）完全可复用
- IR 是编译器架构的支点
- MLOG 是理想的教学编译目标——保留核心、去掉复杂
- 加一个 x86 后端只需要一个额外文件

12 到过程宏 — 编译时翻译的通用框架

 实验记录： 第十一站： `proc macro` 作为编译器前端

前两本教程 (`proc-macro-guide` 和 `rust2mlog`) 都在做同一件事：用过程宏做代码生成。这一章从编译理论的视角重新审视——为什么过程宏是编译器前端的理想载体。

12.1 过程宏本质上是编译器前端

```
// proc-macro-guide 做的事
#[proc_macro_derive(Builder)]
pub fn derive_builder(input: TokenStream) -> TokenStream {
    // input: 结构体定义 (源语言)
    // output: Builder 实现 (目标语言)
    // 两者都是 Rust 代码
}

// rust2mlog 做的事
#[proc_macro]
pub fn mlog(input: TokenStream) -> TokenStream {
    // input: MLOG DSL (源语言)
    // output: MLOG 文本 (目标语言)
    // 源是类 Rust DSL, 目标是 MLOG
}

// 本质上
// TokenStream → 解析 → IR → 代码生成 → TokenStream
//                               ↑ 编译器前端
```

概念

过程宏提供了一个受管理的编译阶段：

- 输入是 `TokenStream` (编译器已经做了词法分析)
- 运行在编译器进程中 (可以访问文件系统、环境变量等)
- 输出自动集成到构建流程中 (不需要额外工具)

12.2 和传统编译器的对比

方面	传统编译器 (<code>gcc/llc</code>)	过程宏编译器	<code> :- :- :- </code>	运行时机	构建时 (独立进程)	构建时 (编译器进程内)
输入	文件路径	<code>TokenStream</code>		输出	<code>.o / .s</code> 文件	<code>TokenStream</code>
集成	需要构建系统配置	自动 (<code>cargo</code>)		错误报告	<code>stderr</code> + 退出码	<code>syn::Error</code> → 编译错误
使用方式	命令行	宏调用				

过程宏不是独立编译器——它是嵌入在 `Rust` 编译器中的翻译函数。这让它和宿主语言有更紧密的集成，但也限制了它的独立性。

12.3 适合放过程宏的场景

适合	不适合	<code> :- :- </code>	代码生成 (<code>Builder</code> , <code>Serialize</code>)	大型独立编译 (>10K 行)
			内嵌 DSL (<code>SQL</code> , <code>HTML</code> , <code>MLOG</code>)	需要独立二进制工具链
			编译时验证 (正则、格式串)	
			非常复杂的优化 <code>pass</code>	协议/配置编译
			需要跨语言共享的	

核心直觉

判断标准：生成的代码是否需要和宿主 `Rust` 代码紧密集成？

如果需要 (`Builder` 模式、`serde`、`sqlx`) → `proc macro` 如果不需要 (独立程序、另一个语言) → 独立编译器

MLOG 编译器确实可以做成独立工具——但做成 `proc macro` 的好处是：它和 Rust 项目中的其他代码共享同一个构建流程。

你用 `cargo build`, MLOG 自动生成。

12.4 任何源语言 → 任何目标语言

过程宏的模式是通用的翻译框架：

```
源 TokenStream → [解析] → [IR] → [代码生成] → 目标 TokenStream
```

源和目标可以是：

- Rust → Rust (`serde`, `thiserror`)
- DSL → Rust (`html!`, `sqlx::query!`)
- DSL → MLOG (`rust2mlog`)
- DSL → SQL (ORM 宏)
- DSL → OpenGL Shader (`shader!` 宏)
- 配置格式 → Rust 结构体

只要你能把源语言解析为 AST，能把目标语言表示为 `TokenStream`，过程宏就能做这个翻译。

12.5 过程宏的局限性

⚠ 注意

过程宏不适合的场景：

1. 极大的编译任务 – 过程宏在编译期运行，慢的宏拖慢所有 `crate` 的编译
2. 需要持久化状态 – 宏不能跨 `crate` 共享数据
3. 需要精确的错误恢复 – `syn::Error` 只能报告，不能修复
4. 需要多 **pass** 分析 – 过程宏是单 **pass** 的（输入 → 输出）

对于 MLOG 编译器来说，这些限制都不是问题——1000 行的编译器在宏中运行毫秒级，不持久化，不跨 `crate`。

12.6 小结

- 过程宏本质上是嵌入在 Rust 编译器中的编译器前端
- `TokenStream` → 解析 → IR → 代码生成 → `TokenStream`
- 适合代码生成、嵌入式 DSL、编译时验证
- 不适合大型独立编译、持久化状态、多 **pass**
- 任何「源语言 → 目标语言」的翻译都可以用 `proc macro` 实现
- 编译理论的通用性 = 过程宏应用场景的通用性

13 翻译链的普遍性

🔗 实验记录：终点站

13.1 三条链，一个骨架

把三本教程放在一起看：

层	proc-macro-guide	rust2mlog	compilation-theory
1	TokenStream	MLOG DSL	C / Rust
2	syn 解析 → DeriveInput	手写解析 → AST	解析 → AST
3	quote! 生成代码	三地址码 IR	三地址码 / LLVM IR
4	impl Trait 块	MLOG 文本	x86 / ARM 汇编
5	编译器拼接	字符串字面量	机器码

🔍 核心直觉

三本教程从三个角度覆盖了同一个问题：如何让机器写代码。

- **proc-macro-guide**: 工具层——过程宏的 API 和模式
- **rust2mlog**: 实践层——用 `proc macro` 实现编译器
- **compilation-theory** (本书): 理论层——编译器为什么这样工作

三者构成一个完整的知识闭环。

13.2 核心概念回顾

| 概念 | 首次出现 | 本质 | |:-|:-|:-| | TokenStream | proc-macro-guide Ch2 | 编译器的“原材料” | | AST | rust2mlog Ch4 | 源代码的树形表示 | | 三地址码 | rust2mlog Ch5 / 本书 Ch6 | 编译器的“通用语言” | | 寄存器分配 | 本书 Ch5 | 有限资源的优雅分配 | | 栈帧 / 调用约定 | 本书 Ch4 | 函数调用的底层机制 | | 控制流图 | 本书 Ch3 | 分析和优化的基础 | | SSA | 本书 Ch6 | 更严格的三地址码 | | 优化 pass | 本书 Ch9 | 不改语义，改善性能 | | proc macro 前端 | 本书 Ch11 | 编译时翻译的框架 |

13.3 编译器设计的核心原则

1. **分阶段**: 解析 → IR → 优化 → 代码生成。每层解决一个问题。
2. **IR 是支点**: 前端和后端的唯一接口。IR 设计好，前后端独立演化。
3. **三地址码是黄金表示**: 足够表达、足够简单、适合优化。
4. **寄存器分配是最难的优化**: NP 完全，用启发式近似。
5. **优化必须保守**: 不能改变可观察行为。
6. **目标架构的约束决定编译器复杂度**: MLOG 无寄存器压力 → 编译器简单; x86 寄存器有限 → 编译器复杂。

13.4 推荐阅读

- **Engineering a Compiler** (Cooper & Torczon) — 编译器全栈教材
- **LLVM Essentials** — 理解 LLVM IR 和优化 pass
- **Crafting Interpreters** (Nystrom) — 解析器和编译器入门
- **Mindcode 源码** — 另一个 MLOG 编译器
- **proc-macro-guide** (本系列) — 过程宏实践
- **rust2mlog** (本系列) — MLOG 编译器实战

13.5 最后的话

三个 PDF，一个主题：代码生成。

从 Rust 过程宏的 API，到 MLOG 编译器的实现，到编译理论的底层原理——三本教程覆盖了从「怎么用」到「怎么做」到「为什么」的完整链条。

当你理解了 $C \rightarrow x86$ 的翻译，你就理解了 $Rust \rightarrow LLVM$ 的翻译。当你理解了 $MLOG \rightarrow$ 三地址码的映射，你就理解了任何编译目标的选择。

编译器是为人类服务的翻译器。过程宏是这个翻译器在 Rust 生态中的具现。

在 Rust 的世界里，只要你能 parse 它，你就能生成它。

而且你现在知道了**为什么**。

